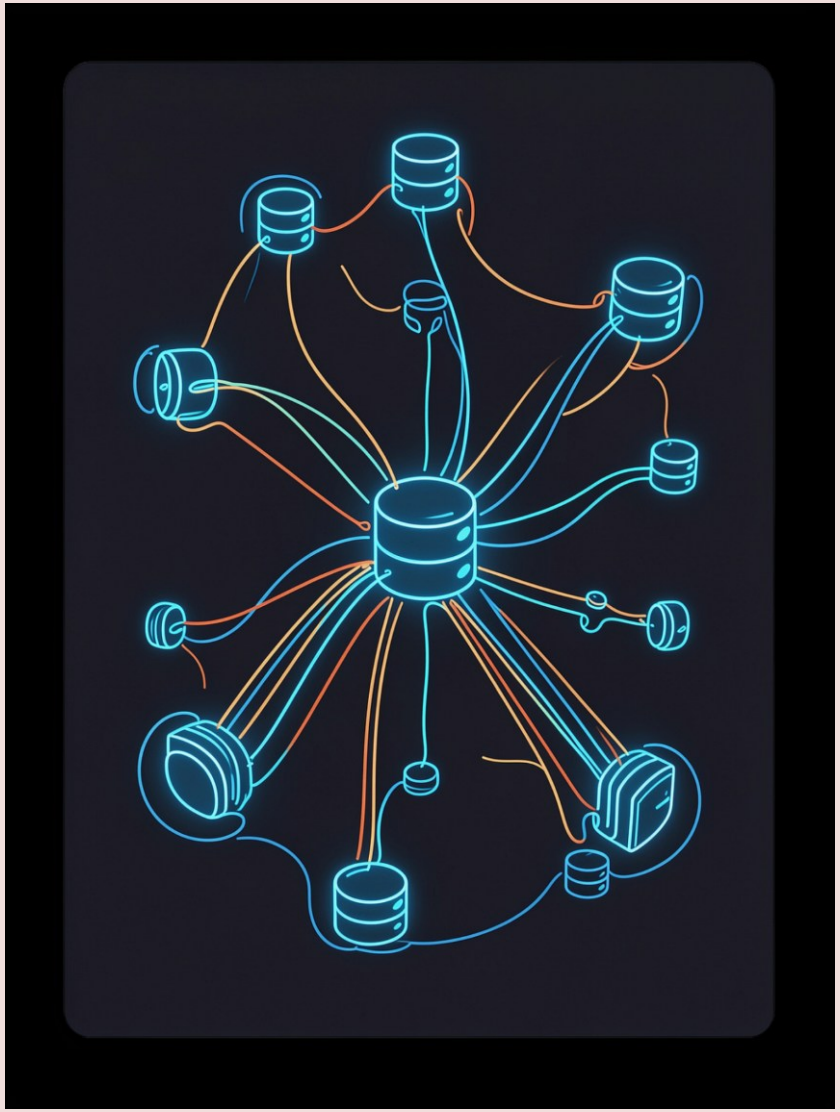
SQL Avanzado IV

Triggers · WITH · WITH Recursive · Funciones Window

MÓDULO AVANZADO

BASES DE DATOS ACTIVAS



Bases de Datos Activas y el Modelo E-C-A

Un DBMS activo incluye un **Motor de Reglas** que ejecuta acciones automáticamente según el modelo **Evento – Condición – Acción**.

1

Evento

INSERT, DELETE, UPDATE; eventos temporales o definidos por el usuario

2

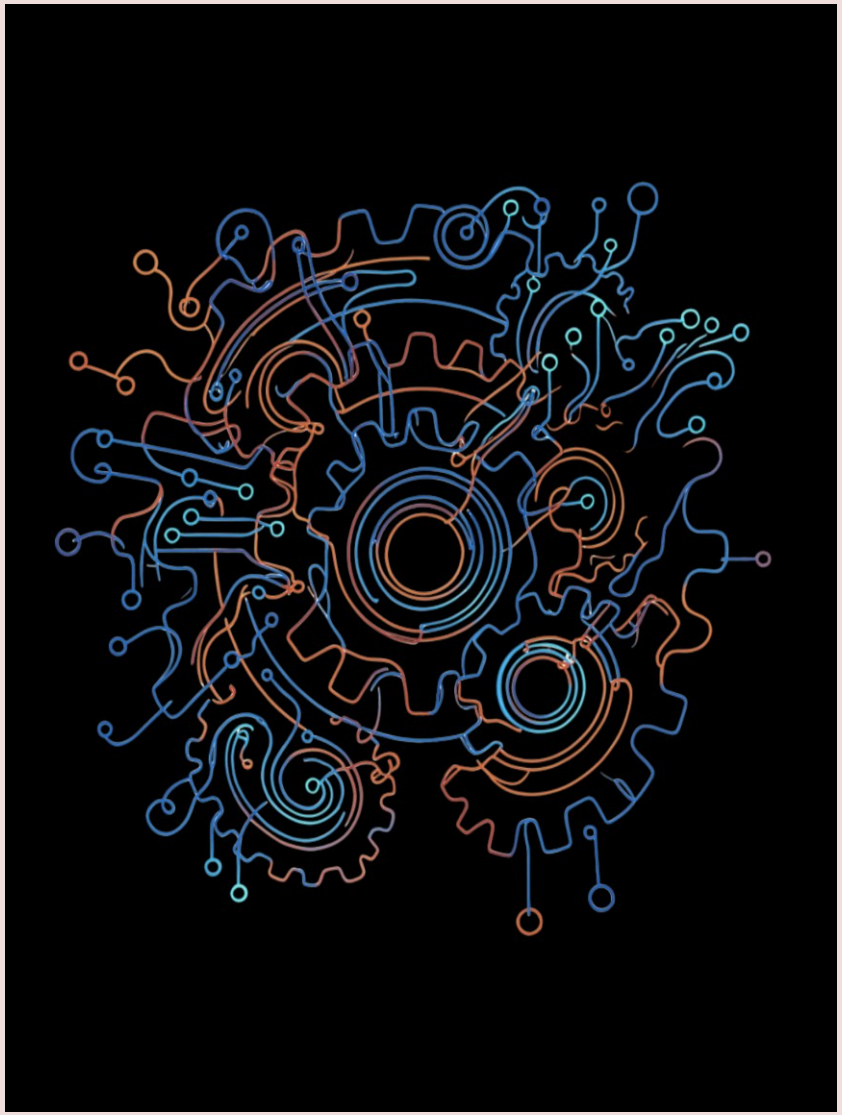
Condición

Predicado SQL opcional que filtra cuándo se ejecuta la acción

3

Acción

Secuencia de sentencias SQL que se ejecutan automáticamente



Triggers: Objetivos y Beneficios

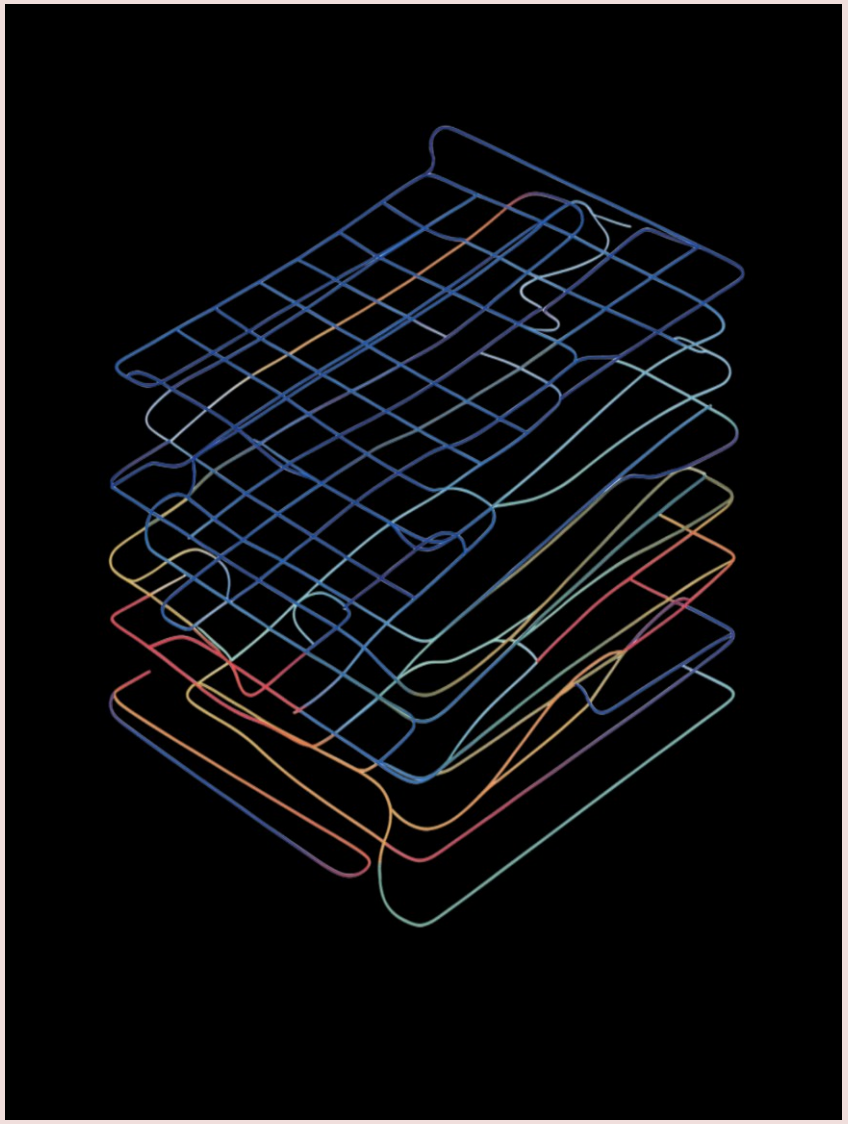


¿Para qué sirven?

- Respetar reglas de integridad
- Generar datos derivados
- Controlar la seguridad
- Implementar reglas de negocio
- Enviar notificaciones y mantener índices

Beneficios clave

- Reglas compartidas por todos los usuarios, sin replicar en cada aplicación
- Ejecución automática, sin intervención del usuario
- Potentes controles de integridad definidos en el esquema



Triggers: Granularidad y Funcionalidad

Por Fila (ROW)

Se activa **una vez por cada tupla** afectada. Más fácil de escribir, menos eficiente.

Por Sentencia (STATEMENT)

Se activa **una sola vez** por sentencia SQL. Más eficiente, más complejo.

Inmediata

BEFORE (antes del evento), **AFTER** (después) o **INSTEAD OF** (en lugar del evento).

Diferida

Se ejecuta al finalizar la transacción, luego del **COMMIT**.

BEFORE vs AFTER: En un trigger BEFORE, los cambios aún no están en la tabla. Ej: con 10 docentes, `SELECT COUNT(*)` devuelve 10 en BEFORE y 11 en AFTER.

Triggers en PostgreSQL: Sintaxis

Estructura CREATE TRIGGER

```
CREATE TRIGGER nombre
  {BEFORE | AFTER}
  {INSERT | UPDATE | DELETE}
  ON tabla
  [FOR EACH {ROW | STATEMENT}]
  [WHEN (condicion)]
  EXECUTE PROCEDURE funcion()
```

Función asociada

```
CREATE OR REPLACE FUNCTION
  "agregarCadena"()
RETURNS trigger AS $BODY$
BEGIN
  INSERT INTO "NuevosArticulos"
    VALUES (NEW."Descripcion");
END;
$BODY$ LANGUAGE 'plpgsql';
```

Claves en PostgreSQL

- El SP debe definirse **antes** que el trigger
- Debe devolver tipo **trigger**
- Tuplas virtuales: **NEW** y **OLD**
- Row-level BEFORE: retornar NULL cancela la operación; retornar NEW la ejecuta
- Row-level AFTER: ignora el valor de retorno
- Statement-level: siempre retornar NULL
- Múltiples triggers se ejecutan en **orden alfabético**
- Variable **TG_OP** indica el evento ('INSERT', 'UPDATE', 'DELETE')
- Posible **cascada**: un trigger puede activar otros (evitar recursión infinita)

Ejemplos prácticos de triggers

Los siguientes ejemplos muestran patrones comunes en PostgreSQL: auditoría, validación de reglas, actualización de datos derivados e interacción con vistas.

1) Trigger de auditoría: registrar cambios en una tabla

Este ejemplo guarda cada inserción, actualización y eliminación en una tabla de auditoría.

```
CREATE TABLE productos (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL,
  precio NUMERIC(10,2) NOT NULL
);

CREATE TABLE productos_auditoria (
  id SERIAL PRIMARY KEY,
  producto_id INT NOT NULL,
  operacion TEXT NOT NULL,
  fecha TIMESTAMP NOT NULL DEFAULT NOW(),
  usuario TEXT NOT NULL DEFAULT CURRENT_USER,
  valor_anterior JSONB,
  valor_nuevo JSONB
);
```

```
CREATE OR REPLACE FUNCTION fn_auditar_productos()
RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO productos_auditoria (producto_id, operacion, valor_nuevo)
    VALUES (NEW.id, TG_OP, to_jsonb(NEW));
    RETURN NEW;

  ELSIF TG_OP = 'UPDATE' THEN
    INSERT INTO productos_auditoria (producto_id, operacion, valor_anterior, valor_nuevo)
    VALUES (NEW.id, TG_OP, to_jsonb(OLD), to_jsonb(NEW));
    RETURN NEW;

  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO productos_auditoria (producto_id, operacion, valor_anterior)
    VALUES (OLD.id, TG_OP, to_jsonb(OLD));
    RETURN OLD;
  END IF;

  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditar_productos
AFTER INSERT OR UPDATE OR DELETE ON productos
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_productos();
```

Ejemplos prácticos de triggers

Los siguientes ejemplos muestran patrones comunes en PostgreSQL: auditoría, validación de reglas, actualización de datos derivados e interacción con vistas.

2) Trigger de reglas de negocio: evitar datos inválidos

```
CREATE TABLE pedidos (
  id SERIAL PRIMARY KEY,
  cliente_id INT NOT NULL,
  total NUMERIC(10,2) NOT NULL,
  estado TEXT NOT NULL
);

CREATE OR REPLACE FUNCTION fn_validar_pedido()
RETURNS trigger AS $$
BEGIN
  IF NEW.total < 0 THEN
    RAISE EXCEPTION 'El total no puede ser negativo';
  END IF;

  IF NEW.estado NOT IN ('pendiente', 'pagado', 'cancelado') THEN
    RAISE EXCEPTION 'Estado inválido: %', NEW.estado;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_pedido
BEFORE INSERT OR UPDATE ON pedidos
FOR EACH ROW
EXECUTE FUNCTION fn_validar_pedido();
```

Ejemplos prácticos de triggers

Los siguientes ejemplos muestran patrones comunes en PostgreSQL: auditoría, validación de reglas, actualización de datos derivados e interacción con vistas.

3) Trigger para mantener datos derivados: actualizar un resumen

Este trigger mantiene una tabla de totales por cliente cada vez que cambia un pedido.

```
CREATE OR REPLACE FUNCTION fn_actualizar_resumen_clientes()
RETURNS trigger AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    INSERT INTO clientes_resumen (cliente_id, total_pedidos,
cantidad_pedidos)
    VALUES (NEW.cliente_id, NEW.total, 1)
    ON CONFLICT (cliente_id) DO UPDATE
      SET total_pedidos = clientes_resumen.total_pedidos + NEW.total,
          cantidad_pedidos = clientes_resumen.cantidad_pedidos + 1;
    RETURN NEW;

  ELSIF TG_OP = 'UPDATE' THEN
    UPDATE clientes_resumen
      SET total_pedidos = total_pedidos - OLD.total + NEW.total
    WHERE cliente_id = NEW.cliente_id;
    RETURN NEW;

  ELSIF TG_OP = 'DELETE' THEN
    UPDATE clientes_resumen
      SET total_pedidos = total_pedidos - OLD.total,
          cantidad_pedidos = cantidad_pedidos - 1
    WHERE cliente_id = OLD.cliente_id;
    RETURN OLD;
  END IF;

  RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TABLE clientes_resumen (
  cliente_id INT PRIMARY KEY,
  total_pedidos NUMERIC(10,2) NOT NULL DEFAULT 0,
  cantidad_pedidos INT NOT NULL DEFAULT 0
);

CREATE TRIGGER trg_resumen_clientes
AFTER INSERT OR UPDATE OR DELETE ON pedidos
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_resumen_clientes();
```

Ejemplos prácticos de triggers

Los siguientes ejemplos muestran patrones comunes en PostgreSQL: auditoría, validación de reglas, actualización de datos derivados e interacción con vistas.

4) Trigger INSTEAD OF para vistas

Los triggers INSTEAD OF permiten insertar o actualizar una vista compleja como si fuera una tabla.

```
CREATE VIEW vista_autores_libros AS
SELECT a.id AS autor_id, a.nombre, l.id AS libro_id, l.titulo
FROM autores a
LEFT JOIN libros l ON l.autor_id = a.id;

CREATE OR REPLACE FUNCTION fn_insertar_vista_autores_libros()
RETURNS trigger AS $$
DECLARE
  nuevo_autor_id INT;
BEGIN
  INSERT INTO autores (nombre)
  VALUES (NEW.nombre)
  RETURNING id INTO nuevo_autor_id;

  INSERT INTO libros (autor_id, titulo)
  VALUES (nuevo_autor_id, NEW.titulo);

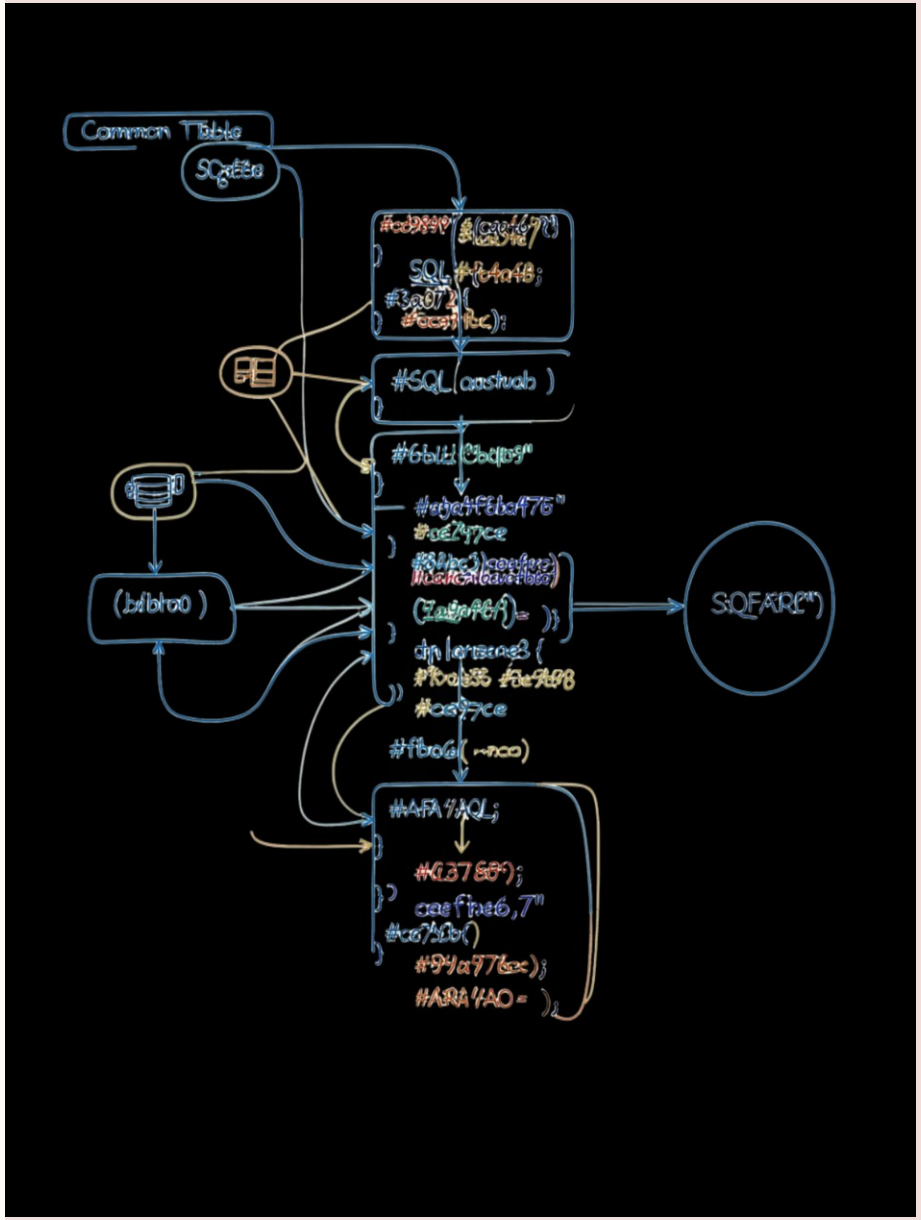
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_insertar_vista_autores_libros
INSTEAD OF INSERT ON vista_autores_libros
FOR EACH ROW
EXECUTE FUNCTION fn_insertar_vista_autores_libros();
```

```
CREATE TABLE autores (
  id SERIAL PRIMARY KEY,
  nombre TEXT NOT NULL
);

CREATE TABLE libros (
  id SERIAL PRIMARY KEY,
  autor_id INT NOT NULL REFERENCES autores(id),
  titulo TEXT NOT NULL
);
```

Nota: en triggers para vistas, **INSTEAD OF** reemplaza la operación sobre la vista y suele usarse para repartir el cambio entre varias tablas base.



WITH: Common Table Expressions (CTE)

Permite definir **consultas precalculadas** reutilizables dentro del cuerpo principal de una consulta, simplificando consultas complejas sin necesidad de vistas temporales.

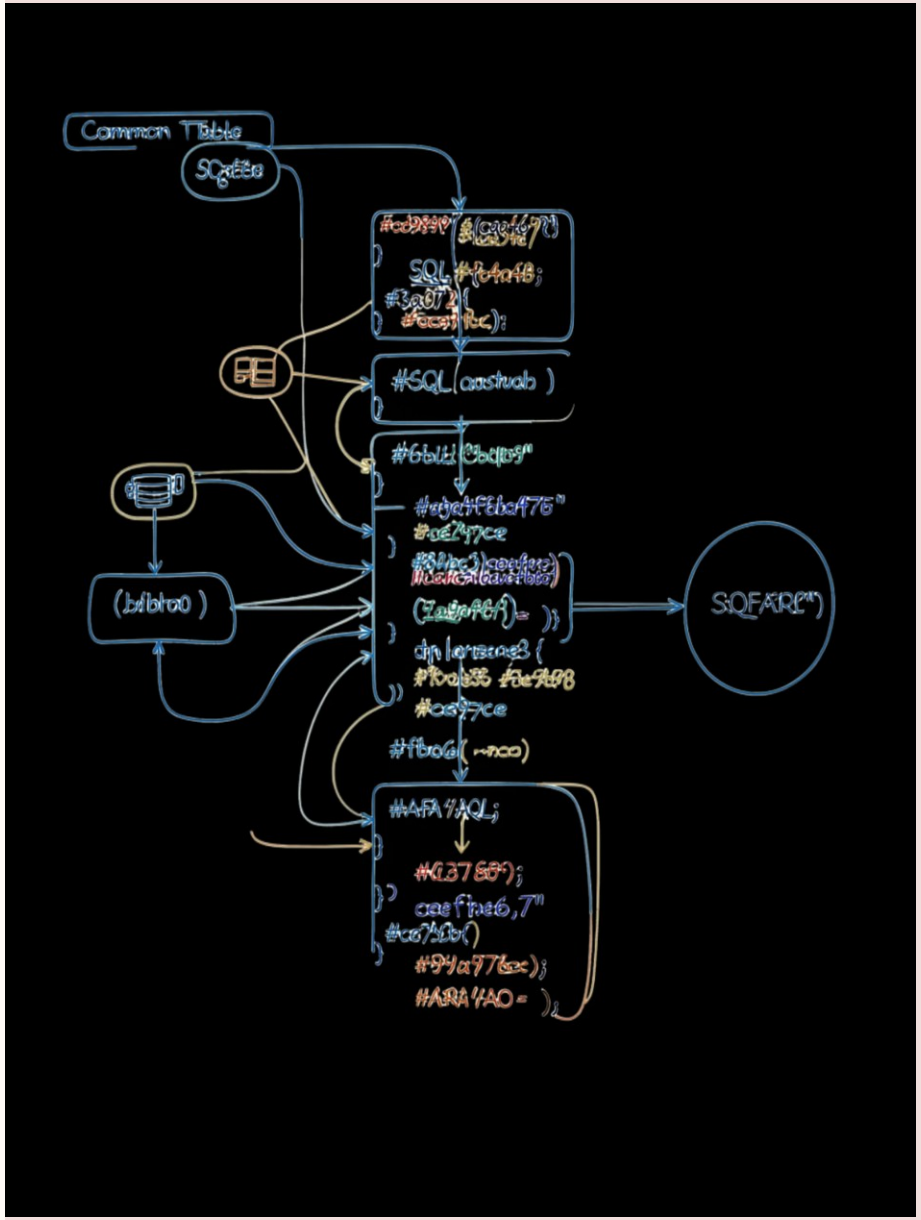
CTE con SELECT

```
WITH horasfaltadas(alumno, horas) AS (
  SELECT i.alumno, SUM(c.horasxclase)
  FROM inscripto i JOIN curso c ...
  GROUP BY i.alumno)
SELECT * FROM horasfaltadas hf
JOIN persona p ON p.persona_id = hf.alumno;
```

CTE para totales acumulados

Útil para reportes financieros, seguimiento de ventas o métricas por día.

```
WITH ventas_diarias AS (
  SELECT
    fecha::date AS dia,
    SUM(monto) AS ventas_dia
  FROM ventas
  GROUP BY fecha::date
),
acumulado AS (
  SELECT
    dia,
    ventas_dia,
    SUM(ventas_dia) OVER (ORDER BY dia) AS total_acumulado
  FROM ventas_diarias
)
SELECT *
FROM acumulado
ORDER BY dia;
```



WITH: Common Table Expressions (CTE)

Permite definir **consultas precalculadas** reutilizables dentro del cuerpo principal de una consulta, simplificando consultas complejas sin necesidad de vistas temporales.

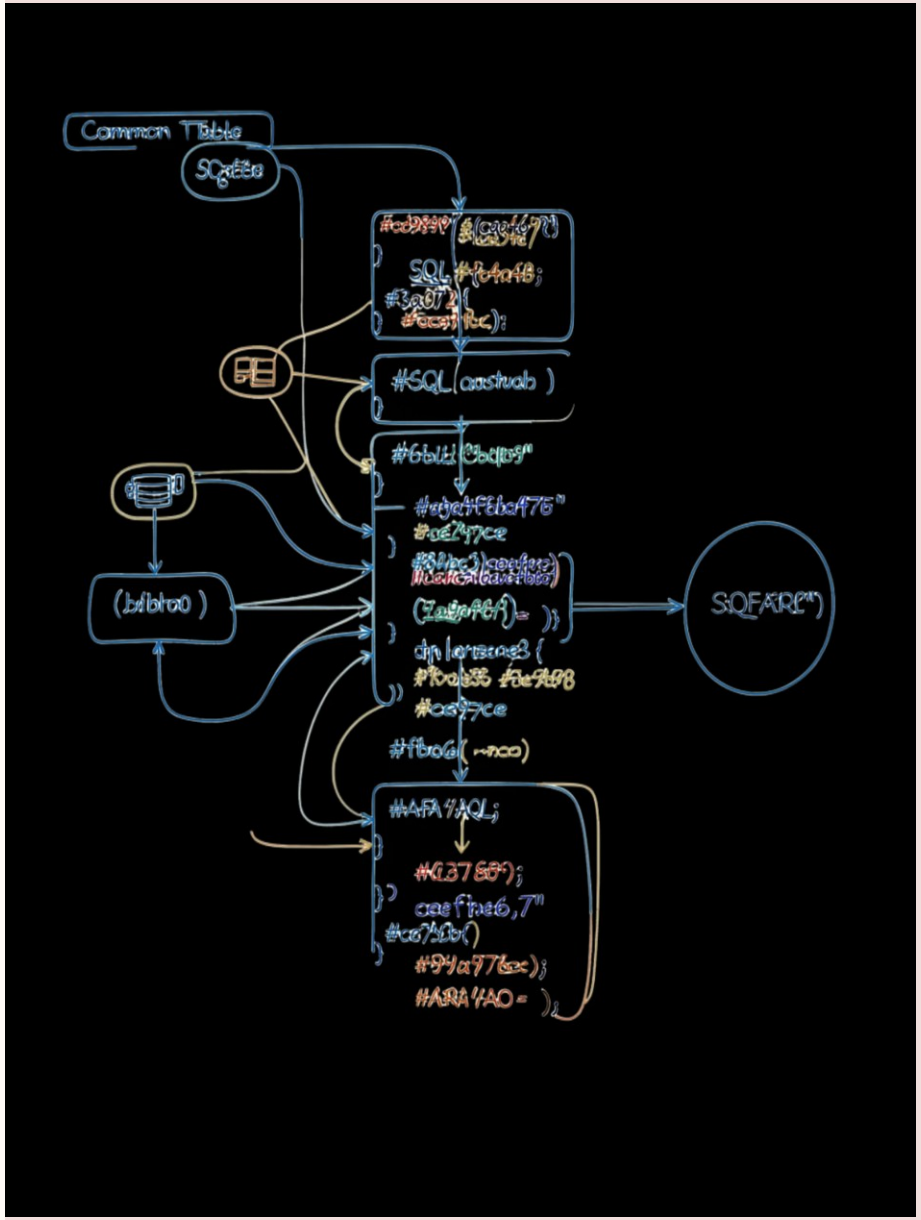
CTE con DELETE / INSERT

```
WITH moved_rows AS (
 DELETE FROM products
 WHERE "date" BETWEEN '2010-10-01' AND '2010-11-01'
 RETURNING *)
INSERT INTO products_log SELECT * FROM moved_rows;
```

Múltiples CTE encadenados

Divide una lógica compleja en pasos claros y reutilizables.

```
WITH pedidos_filtrados AS (
  SELECT id, cliente_id, total
  FROM pedidos
  WHERE estado = 'confirmado'
),
totales_cliente AS (
  SELECT
    cliente_id,
    COUNT(*) AS cantidad_pedidos,
    SUM(total) AS gasto_total
  FROM pedidos_filtrados
  GROUP BY cliente_id
),
clientes_premium AS (
  SELECT *
  FROM totales_cliente
  WHERE gasto_total > 1000
)
SELECT c.nombre, p.cantidad_pedidos, p.gasto_total
FROM clientes_premium p
JOIN clientes c ON c.id = p.cliente_id
ORDER BY p.gasto_total DESC;
```



WITH: Common Table Expressions (CTE)

Permite definir **consultas precalculadas** reutilizables dentro del cuerpo principal de una consulta, simplificando consultas complejas sin necesidad de vistas temporales.

CTE con agregación y filtro

Ideal para resumir datos y luego aplicar criterios de negocio sobre el resumen.

```
WITH resumen_departamentos AS (
  SELECT
    d.nombre AS departamento,
    COUNT(e.id) AS cantidad_empleados,
    AVG(e.salario) AS salario_promedio
  FROM departamentos d
  JOIN empleados e ON e.departamento_id = d.id
  GROUP BY d.nombre
)
SELECT *
FROM resumen_departamentos
WHERE cantidad_empleados >= 10
  AND salario_promedio > 50000
ORDER BY salario_promedio DESC;
```

CTE con UPDATE

Permite actualizar filas usando primero un conjunto intermedio calculado.

```
WITH clientes_activos AS (
  SELECT id
  FROM clientes
  WHERE ultimo_pedido > CURRENT_DATE - INTERVAL '90 days'
)
UPDATE clientes c
SET estado = 'activo'
FROM clientes_activos a
WHERE c.id = a.id;
```

WITH RECURSIVE

¿Cuándo usarlo?

Para consultas con **estructuras jerárquicas o recursivas** (árboles, grafos, clausuras transitivas) que no se resuelven con SQL estándar.

Ejemplo: árbol de empleados

```
WITH RECURSIVE distanciaDesdeRaiz
  (persona_id, dependede, distancia) AS (
  SELECT persona_id, dependede, 0
  FROM empleado WHERE dependede IS NULL
  UNION ALL
  SELECT e.persona_id, e.dependede,
         dr.distancia + 1
  FROM distanciaDesdeRaiz dr, empleado e
  WHERE dr.persona_id = e.dependede)
SELECT * FROM distanciaDesdeRaiz;
```

Algoritmo de evaluación

01

Evaluar el término **no recursivo** (caso base)

02

Descartar duplicados (solo con **UNION**, no UNION ALL)

03

Incluir filas en resultado y copiarlas a la **TTT**

04

Mientras TTT no esté vacía: evaluar término recursivo, descartar duplicados, mover a TIT y reemplazar TTT

Más ejemplos prácticos

1) Descendientes en una jerarquía

Encuentra todos los subordinados de un jefe dado recorriendo la jerarquía hacia abajo.

```
WITH RECURSIVE subordinados AS (
  SELECT empleado_id, nombre, jefe_id, 0 AS nivel
  FROM empleados
  WHERE empleado_id = 10
UNION ALL
  SELECT e.empleado_id, e.nombre, e.jefe_id, s.nivel + 1
  FROM empleados e
  JOIN subordinados s ON e.jefe_id = s.empleado_id
)
SELECT *
FROM subordinados
ORDER BY nivel, empleado_id;
```

2) Generar una secuencia o serie

Útil para crear rangos de fechas, números consecutivos o calendarios simples sin tabla auxiliar.

```
WITH RECURSIVE numeros AS (
  SELECT 1 AS n

  UNION ALL

  SELECT n + 1
  FROM numeros
  WHERE n < 10
)
SELECT n
FROM numeros;
```

WITH RECURSIVE

3) Búsqueda de rutas o recorrido de grafos

Explora conexiones entre nodos y construye la ruta completa acumulando el camino recorrido.

```
WITH RECURSIVE rutas AS (
  SELECT origen, destino, ARRAY[origen, destino] AS camino
  FROM aristas
  WHERE origen = 'A'

  UNION ALL

  SELECT r.origen, e.destino, r.camino || e.destino
  FROM rutas r
  JOIN aristas e ON r.destino = e.origen
  WHERE NOT e.destino = ANY(r.camino)
)
SELECT *
FROM rutas;
```

4) Recursión con límite de profundidad

Limita la expansión para evitar recorridos demasiado grandes o ciclos muy profundos.

```
WITH RECURSIVE jerarquia AS (
  SELECT id, padre_id, 0 AS profundidad
  FROM categorias
  WHERE padre_id IS NULL

  UNION ALL

  SELECT c.id, c.padre_id, j.profundidad + 1
  FROM categorias c
  JOIN jerarquia j ON c.padre_id = j.id
  WHERE j.profundidad < 3
)
SELECT *
FROM jerarquia;
```

Funciones Window

Las funciones window permiten calcular **agregaciones, rankings y comparaciones** sobre particiones del conjunto resultado **sin agrupar** las filas, preservando el detalle individual.

1) ROW_NUMBER() para ranking por departamento

Ejemplo completo con datos de prueba y el resultado esperado ordenado por departamento y salario.

```
WITH empleados AS (
  SELECT * FROM (VALUES
    ('Ventas',  'Ana',   5200),
    ('Ventas',  'Bruno', 6100),
    ('Ventas',  'Carla', 6100),
    ('TI',      'Diego', 7000),
    ('TI',      'Elena', 6800),
    ('TI',      'Fabio', 7200)
  ) AS t(departamento, empleado, salario)
)
SELECT
  departamento,
  empleado,
  salario,
  ROW_NUMBER() OVER (
    PARTITION BY departamento
    ORDER BY salario DESC, empleado
  ) AS ranking
FROM empleados
ORDER BY departamento, ranking;
```

departamento	empleado	salario	ranking	nota
TI	Fabio	7200	1	Mayor salario en TI
TI	Diego	7000	2	Segundo puesto
TI	Elena	6800	3	Tercer puesto
Ventas	Bruno	6100	1	Desempate por nombre
Ventas	Carla	6100	2	

Funciones Window

2) LAG() y LEAD() para comparar ventas mes a mes

Este ejemplo muestra la venta anterior, la siguiente y las diferencias respecto al mes previo y al siguiente.

```
WITH ventas_mensuales AS (
  SELECT * FROM (VALUES
    ('2024-01', 120),
    ('2024-02', 150),
    ('2024-03', 140),
    ('2024-04', 180),
    ('2024-05', 210)
  ) AS t(mes, ventas)
)
SELECT
  mes,
  ventas,
  LAG(ventas) OVER (ORDER BY mes) AS ventas_mes_anterior,
  ventas - LAG(ventas) OVER (ORDER BY mes) AS diferencia_vs_anterior,
  LEAD(ventas) OVER (ORDER BY mes) AS ventas_mes_siguiente,
  LEAD(ventas) OVER (ORDER BY mes) - ventas AS diferencia_vs_siguiente
FROM ventas_mensuales
ORDER BY mes;
```

mes	ventas	ventas_mes_anterior	diferencia_vs_anterior	ventas_mes_siguiente	diferencia_vs_siguiente
2024-01	120			150	30
2024-02	150	120	30	140	-10
2024-03	140	150	-10	180	40
2024-04	180	140	40	210	30
2024-05	210	180	30		

Funciones Window

3) SUM() OVER para acumulado

Usa una ventana acumulativa para ver el total progresivo de ventas a lo largo del tiempo.

```
WITH ventas_mensuales AS (
  SELECT * FROM (VALUES
    ('2024-01', 120),
    ('2024-02', 150),
    ('2024-03', 140),
    ('2024-04', 180),
    ('2024-05', 210)
  ) AS t(mes, ventas)
)
SELECT
  mes,
  ventas,
  SUM(ventas) OVER (
    ORDER BY mes
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS acumulado
FROM ventas_mensuales
ORDER BY mes;
```

mes	ventas	acumulado	nota
2024-01	120	120	Primer mes
2024-02	150	270	Suma hasta febrero
2024-03	140	410	Suma hasta marzo
2024-04	180	590	Suma hasta abril
2024-05	210	800	Suma total acumulada

Funciones Window

4) RANK() vs DENSE_RANK() con empates

Ambas funciones asignan posiciones según el valor, pero **RANK()** deja huecos después de un empate y **DENSE_RANK()** no.

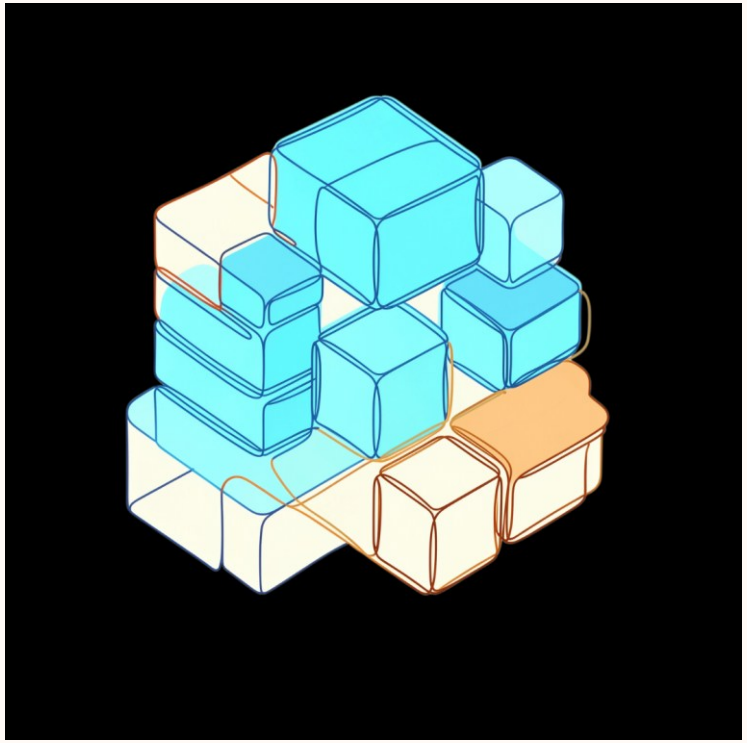
```
WITH puntuaciones AS (
  SELECT * FROM (VALUES
    ('Ana',   95),
    ('Bruno', 90),
    ('Carla', 90),
    ('Diego', 80),
    ('Elena', 75)
  ) AS t(participante, puntos)
)
SELECT
  participante,
  puntos,
  RANK() OVER (ORDER BY puntos DESC) AS rank_olimpico,
  DENSE_RANK() OVER (ORDER BY puntos DESC) AS dense_rank
FROM puntuaciones
ORDER BY puntos DESC, participante;
```

participante	puntos	rank_olimpico	dense_rank	explicación
Ana	95	1	1	Primer lugar
Bruno	90	2	2	Empate en segundo lugar
Carla	90	2	2	Empate en segundo lugar
Diego	80	4	3	RANK() salta el 3; DENSE_RANK() no
Elena	75	5	4	Sigue la secuencia

Vistas Materializadas

¿Qué son?

A diferencia de las vistas comunes (que ejecutan la consulta en cada acceso), las **vistas materializadas** almacenan físicamente el resultado de la consulta, mejorando el rendimiento en consultas costosas y frecuentes.



Comandos clave

- `CREATE MATERIALIZED VIEW` — crea y pobla la vista
- `REFRESH MATERIALIZED VIEW` — actualiza los datos
- `REFRESH MATERIALIZED VIEW CONCURRENTLY` — sin bloquear lecturas

Ideal para reportes, dashboards y consultas agregadas sobre grandes volúmenes de datos.



¿Preguntas?

Repasemos los conceptos clave del módulo:

Triggers

Modelo E-C-A, granularidad, BEFORE/AFTER, PostgreSQL

WITH / CTE

Consultas precalculadas, DELETE/INSERT con RETURNING

WITH RECURSIVE

Jerarquías, árboles, clausuras transitivas

Funciones Window

PARTITION BY, ORDER BY, agregaciones sin GROUP BY